kgzip2: un compressore reversibile in Kairos

Il decompressore come inverso del compressore, e l'ordinamento delle rotazioni in $O(n \log n)$

Nicolò Giuliani

Università di Bologna, relazione tecnica, ottobre 2026

Sommario

In un linguaggio reversibile comprimere e decomprimere non sono due programmi da tenere allineati: il decompressore è il compressore eseguito all'indietro. Questo lavoro estende il caso di studio del capitolo 4 della tesi (trasformata di Burrows e Wheeler reversibile, a blocchi paralleli, dopo Lyngby et al. [1]) a un compressore completo, *kgzip2*, costruito sulla pipeline di bzip2: RLE iniziale, BWT, move-to-front, RLE degli zeri e codice di Elias gamma. Tutto è una sola procedura Kairos, `comprimi`. Il programma di decompressione riceve soltanto i bit compressi e cinque contatori per blocco ed esegue `uncall comprimi(...)`: nel sorgente non esiste una riga di decodifica. Perché questo funzioni ogni stadio è *pulito*: consuma l'ingresso e lascia solo l'uscita, compresa la BWT, che nel capitolo 4 lasciava il blocco originale. Il secondo contributo riguarda il punto che [1] indica come miglioramento necessario, l'ordinamento delle rotazioni: dal bubble sort $O(n^3)$ si passa a un confronto con uscita anticipata, a un merge sort reversibile e infine a un ordinamento per raddoppio dei prefissi, $O(n \log n)$ qualunque siano i dati. Su 9 file il ritorno è identico byte per byte. Sul testo, con blocchi da 8 KiB, kgzip2 arriva a 44,0%–47,9% della dimensione originale (bzip2: 39,2%–42,4%) e sui dati molto regolari fa meglio di bzip2; ordinare un blocco periodico da 64 byte passa da 27,4 s a 0,07 s, e uno da 8192 byte richiede 10,5 s. L'eseguibile `kgzip2` è un file unico che funziona senza Kairos installato e lavora in streaming, a gruppi di blocchi: la memoria resta costante qualunque sia la dimensione del file. Infine, un file alterato che sfugga al CRC quasi sempre ferma la VM da sola: le condizioni che rendono il programma invertibile, lette all'indietro, diventano controlli dell'ingresso che nessuno ha scritto.

1 Introduzione

La compressione lossless è il banco di prova naturale per un linguaggio reversibile. Comprimere e decomprimere sono l'una l'inversa dell'altra, quindi in un linguaggio in cui ogni programma ha un inverso calcolabile basta scriverne una metà, e le due metà non possono divergere. Il prezzo è che ogni passo deve conservare l'informazione: niente cancellazioni, niente variabili sovrascritte, niente uscite anticipate da un `if`.

Lyngby, Nylandsted, Glück e Yokoyama [1] hanno scritto in Janus versioni reversibili e *pulite* di due algoritmi dello standard zip, LZW e la BWT. La loro BWT ordina le rotazioni con un algoritmo ingenuo, $O(n^3)$, e il lavoro conclude che migliorare quell'ordinamento è possibile e necessario. Il loro lavoro è sequenziale. Il capitolo 4 della tesi ha portato la BWT in Kairos e ha trasformato i blocchi in parallelo, come rami di un `par`.

Questo lavoro fa il passo successivo: un compressore intero, da file a file, con la decompressione ottenuta per inversione. I contributi:

1. **kgzip2**, una pipeline nello stile di bzip2 [7] (RLE, BWT, MTF, RLE degli zeri, codice a prefisso) scritta come un'unica procedura pulita; la decompressione è `uncall` della stessa procedura;
2. la **BWT pulita in senso stretto**: anche l'ingresso viene azzerato, che era uno dei limiti dichiarati del capitolo 4;
3. una sequenza di **ordinamenti reversibili delle rotazioni** che porta il costo da $O(n^3)$ a $O(n \log n)$ indipendentemente dai dati, con le tecniche che servono a renderli reversibili;

4. una **valutazione**: correttezza byte per byte, rapporto di compressione contro bzip2, tempi, casi peggiori, concorrenza.

2 Contesto

Kairos. Linguaggio reversibile e concorrente della famiglia di Janus [5]: assegnamenti reversibili (`+=`, `-=`, `<=>`), condizionali con asserzione d'uscita (`if` e_1 `then ... fi` e_2), cicli `from` e_1 `do ... until` e_2, blocchi `local`/`delocal`, `call`/`uncall`, array, stack, `par` e canali sincroni. Le guardie sono espressioni con `&&` e `||`; gli stack hanno `push` e `pop` ma non un'operazione per leggere la cima. Il frontend è in Python e produce bytecode testuale; la macchina virtuale, in C, lo esegue nei due versi.

Pulito. Un algoritmo reversibile non può cancellare. Lyngby et al. chiamano *clean* un programma che lascia solo l'uscita. La tecnica è una coppia di procedure: una produce l'uscita e dei dati ausiliari, l'altra ricalcola gli stessi ausiliari partendo dall'uscita; chiamare la prima e disfare la seconda lascia la sola uscita. Per un compressore la pulizia è la condizione perché l'inverso sia un decompressore: se `comprimi` lasciasse spazzatura, il suo inverso avrebbe bisogno anche di quella, che un file compresso non contiene.

Spazzatura alla Bennett. Quando un passo non si può disfare subito, il dato che servirebbe a disfarlo va su uno stack (`scarto`) e sparisce disfacendo l'intera procedura che l'ha prodotto [4]. È lo schema `call ordina; call estrai; uncall ordina`: l'ordinamento lascia spazzatura, l'estrazione usa l'ordine, e l'ordinamento disfatto la porta via.

Testimoni. Ogni `if` deve dire in uscita quale ramo ha preso. Quando la condizione d'ingresso non è più verificabile dopo il corpo, si usa un testimone: una variabile che vale 1 solo se si è entrati, e che poi va azzerata deducendola dallo stato.

3 `kgzip2 file.txt`, passo per passo

Questa sezione descrive l'intero percorso come lo vede chi usa il programma. L'eseguibile esiste: `dist/bin/kgzip2` è un file unico di circa 8 MB, costruito con PyInstaller, che funziona su una macchina senza Kairos né Python. Ha le opzioni di gzip e bzip2: `-d`, `-t`, `-k`, `-c`, `-f`, `-v`, `-1`...`-9`, più `-T` per il numero di blocchi per esecuzione della VM, e senza file fa da filtro fra stdin e stdout. Lavora in streaming: un file di qualunque dimensione passa a gruppi di blocchi, con la memoria costante.

3.1 Che cosa c'è dentro l'eseguibile

- **la libreria Kairos** `kgzip2.kairos`: tutti gli stadi e la procedura `comprimi`;
- **il frontend Kairos** (lexer, parser LALR, controlli statici, generatore di bytecode), che traduce la libreria e un `main` senza dati: poche centinaia di righe, in meno di un decimo di secondo;
- **la macchina virtuale** (`libvm.so`), che esegue il bytecode in avanti o all'indietro e realizza `par` con un thread per ramo;
- **un involucro** che legge e scrive i file in streaming, scrive i dati direttamente come istruzioni di bytecode e gestisce il formato a frame `.kgz`. Non comprime nulla: sposta byte e impacchetta bit.

La VM gira in un processo figlio con un tempo massimo, così un programma che non terminasse non bloccherebbe il comando. La stessa costruzione produce `klzma`.

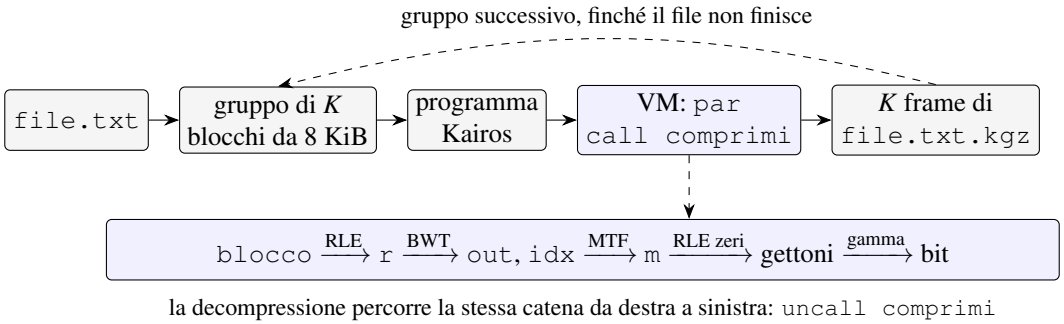


Figura 1: Il percorso di `kgzip2`. In grigio l'involucro, in azzurro ciò che esegue Kairos. Il ciclo si ripete per ogni gruppo di K blocchi (uno per core): in memoria c'è un gruppo alla volta.

3.2 Compressione

1. Lettura a gruppi. Il file si legge come flusso, a blocchi di B byte ($B = 1024 \cdot$livello, 8192 con `-8`); l'ultimo può essere più corto. I blocchi sono indipendenti e si raccolgono in gruppi di K, uno per core (`-T` lo cambia): ogni gruppo è un'esecuzione della VM, con i blocchi come rami di un `par`, e i passi 2–4 si ripetono per ogni gruppo. In memoria ci sono solo i blocchi del gruppo corrente, più uno letto in anticipo per sapere qual è l'ultimo: la RAM non dipende dalla dimensione del file (§5.5), e il comando funziona anche da una pipe, senza conoscere la lunghezza dei dati. Un blocco di meno di 2 byte, che può essere solo l'ultimo, si memorizza senza passare dalla VM.

2. Caricamento. La VM non legge file, quindi i dati entrano nello store del programma. L'involucro genera un `main` che dichiara, per ogni blocco i, gli array di lavoro (tutti a zero) e contiene un segnaposto al posto dei dati. Solo questa parte passa dal frontend; al posto del segnaposto, nel bytecode, l'involucro scrive un'istruzione per byte, `PUSHEQ blocco_0[k] v`, cioè ciò che il frontend produrrebbe da `blocco_0[k] += v`:

```
local int blocco_0[12] = 0
...
blocco_0[0] += 98       // 'b'
blocco_0[1] += 97       // 'a'
...
par
    call comprimi(blocco_0, r_0, ..., bit_0, idx_0, n1_0, ..., nr_0)
and
    call comprimi(blocco_1, r_1, ..., bit_1, idx_1, n1_1, ..., nr_1)
rap
show(idx_0)  show(n1_0)  show(ng_0)  show(nel_0)  show(nb_0)  show(bit_0)
```

Il parser non vede mai i dati: in decompressione, dove i bit sono decine di migliaia, questo toglie circa 5 secondi su 8 KiB. E poiché i gruppi pieni hanno tutti la stessa forma (K blocchi da B byte), il frontend traduce il programma una volta sola: ogni gruppo riusa lo stesso bytecode, con i propri dati al posto del segnaposto. Il trattino basso fra nome e indice non è estetico: senza, il contatore `n1` del blocco 0 e la lunghezza `n` del blocco 10 si chiamerebbero entrambi `n10`, e da 11 blocchi in su il controllo statico di Kairos fermerebbe il programma per una corsa critica nel `par` (è successo alla prima versione, che con blocchi da 8 KiB non comprimeva file oltre 80 KiB).

3. Esecuzione. La VM esegue `call comprimi` su ogni ramo in parallelo. I rami scrivono array disgiunti, quindi il controllo statico sul `par` li accetta. Dentro `comprimi` gli stadi si susseguono; ciascuno ha come precondizione l'uscita a zero e come postcondizione l'ingresso a zero. La Tabella 1 segue un blocco di 12 byte.

stadio	stato dopo lo stadio
ingresso	`barbaaaaara!` = 98 97 114 98 97 97 97 97 97 114 97 33 $(n = 12)$
RLE iniziale	98 97 114 98 **1 0 97 97 97 97** 114 97 33 ($n_1 = 13$: la corsa di cinque `a` diventa $(5-4)$ mod 256, $(5-4)$ div 256, quattro copie)
BWT, ordine	`offs` = 5 4 12 11 6 7 8 9 1 3 0 10 2
BWT, uscita	`out` = 1 98 97 114 0 97 97 97 98 114 33 97 97, `idx` = 10
move-to-front	`m` = 1 98 98 114 4 2 0 0 3 3 36 3 0
RLE degli zeri	1 98 98 114 4 2 **2 0** 3 3 36 3 **1 0** (12 gettoni, 14 elementi)
Elias gamma	84 bit: `101000100101001000000010000100111011010 1...`
frame	84 bit più cinque contatori non sono meno di 12 byte: il blocco si memorizza, e il file è `KGZ4` con un solo frame: tipo (memorizzato, ultimo), lunghezza, i 12 byte, il CRC dei dati e quello del file (26 byte)

Tabella 1: Un blocco attraverso la pipeline. Su un ingresso così corto la codifica non conviene e scatta il blocco memorizzato; i numeri dei passi sono quelli stampati da Kairos e coincidono con il modello Python.

3a. RLE iniziale. Come il primo stadio di bzip2, serve soprattutto a proteggere l'ordinamento dalle corse lunghe. Una corsa di $L \geq 4$ byte uguali a v diventa sei elementi, dal basso: $(L-4)$ mod 256, $(L-4)$ div 256, v, v, v, v; gli altri byte restano se stessi. Rispetto a bzip2 il conteggio sta *sotto* le copie, e il motivo è il decodificatore: `uncall` legge lo stack d'uscita dalla cima, e dalla cima una corsa si riconosce da quattro valori uguali. L'ambiguità non c'è perché le corse sono massimali (dopo una corsa di v viene un byte diverso) e fuori dalle corse un byte si ripete al più tre volte. Il ramo preso si scrive in un testimone `z`, azzerato guardando i quattro elementi in cima:

```
if n1 >= 4 then
    pop(a, out)  pop(b, out)  pop(c, out)  pop(d, out)
    if (a == b) && (b == c) && (c == d) then
        z -= 1
    fi (a == b) && (b == c) && (c == d)
    push(d, out)  push(c, out)  push(b, out)  push(a, out)
fi n1 >= 4
```

Una corsa corta si emette un byte alla volta: si spinge v e le copie restanti tornano sull'ingresso con `uncall corsa`, il contatore della corsa usato al contrario.

3b. BWT. Le rotazioni del blocco si ordinano (§4); l'ultima colonna è `out`, e `idx` è la posizione della rotazione originale. Per la pulizia si applica la coppia di procedure di [1] all'intera trasformata:

```
procedure bwt(...)
    call ordina(offs, blocco, ...)          // offs + spazzatura su scarto
    call estrai(offs, blocco, out, n, idx)  // out, idx
    uncall ordina(offs, blocco, ...)        // offs e scarto tornano vuoti
    uncall bwt_inversa(out, ..., blocco, idx, n, na)
```

`bwt_inversa` porta $(\texttt{out}, \texttt{idx}, 0)$ in $(\texttt{out}, \texttt{idx}, \texttt{blocco})$ lasciando a zero i propri ausiliari; chiamata all'indietro su un blocco pieno, lo ricalcola dalla sola uscita e lo azzera. È il *clean* in senso stretto che il capitolo 4 lasciava aperto. L'inversa usa la tabella $T[i] = \#\{j : \texttt{out}[j] < \texttt{out}[i]\} + \#\{j < i : \texttt{out}[j] = \texttt{out}[i]\}$, costruita con un istogramma e la sua cumulata in $O(n + |\Sigma|)$ invece delle due scansioni $O(n^2)$ del capitolo 4.

3c. Move-to-front. Per ogni simbolo si cerca la sua posizione p nella tabella (con uscita anticipata: il ciclo si ferma su `tab[(j - 1)] == v` e la variabile torna a zero con $j -= p + 1$), si scrive p e si porta il simbolo in testa con una catena di scambi. Dopo lo spostamento il simbolo sta in `tab[0]`, quindi `s[i] -= tab[0]` azzera l'ingresso senza perdere nulla. La tabella finale dipende dai dati, ma solo attraverso le posizioni scritte: `rigioca` ripete gli spostamenti di `m` a partire dall'identità, e `uncall rigioca` la

riporta all'identità.

3d. RLE degli zeri. Dopo BWT e MTF gli zeri dominano. Un valore non nullo resta un gettone; una corsa di L zeri diventa $L, 0$, con lo zero in cima che all'indietro annuncia una lunghezza. Il testimone del ramo si azzera guardando la cima dell'uscita. La scansione della corsa riusa `corsa` del convertitore del capitolo 4, con la sentinella -1 in fondo allo stack d'ingresso.

3e. Codice di Elias gamma. Ogni elemento e diventa il codice gamma [6] di $x = e+1$, in $2\lfloor \log_2 x \rfloor + 1$ bit: i valori piccoli, la maggioranza dopo MTF, costano pochi bit ($x = 1$ ne costa uno). Il dimezzamento è reversibile tenendo il resto:

```
h += x / 2       // h = meta'
x -= h
x -= h           // x = bit meno significativo
push(x, bit)     // x = 0
x <=> h          // si prosegue con la meta'
```

Gli L zeri della parte unaria li spinge `uncall corsa`: all'indietro `call corsa` li consuma e si ferma da sola sull'1 che li precede, senza dover conoscere L. L'ordine dei bit è quello in cui li consuma l'inverso, dalla cima.

4. Raccolta e frame. Alla fine del `par` ogni ramo ha lasciato soltanto lo stack `bit` e cinque contatori: `idx`, n_1 (lunghezza dopo l'RLE), numero di gettoni, numero di elementi, numero di bit. Tutto il resto è a zero e i `delocal` del `main` lo verificano: un ausiliario non azzerato fermerebbe il programma. L'involucro scrive subito un frame per blocco e passa al gruppo successivo. Il file `.kgz` è la firma `KGZ4` seguita dai frame; un frame contiene il tipo (modo 0 Kairos o 1 memorizzato, più l'indicatore dell'ultimo frame), la lunghezza del blocco e i cinque contatori in varint, i bit impacchettati in byte e un CRC32 del file dal primo byte fino a lì. L'ultimo frame porta anche il CRC32 di tutti i dati originali, calcolato man mano. Nessun campo dipende dai blocchi che seguono, quindi il formato si scrive senza tornare indietro, e l'uscita non dipende da K: con `-T 1` o `-T 8` i byte sono gli stessi. Un blocco i cui bit non sono più corti dei suoi byte si memorizza tale e quale (modo 1), come i blocchi non compressi di LZMA2: è il caso dei dati casuali e dei file molto corti.

3.3 Decompressione: `kgzip2 -d file.txt.kgz`

1. Lettura e controllo. Anche la decompressione procede a gruppi di K frame. L'involucro legge un frame e ne controlla il CRC *prima* di avviare la VM: un file alterato si ferma al primo frame danneggiato e non arriva mai all'`uncall`, e un file troncato si riconosce perché manca l'ultimo frame. Poi genera un secondo programma, che dichiara gli stessi array a zero e carica lo stato *finale* della compressione: i bit con istruzioni `PUSH` scritte direttamente nel bytecode, i contatori con `PUSHEQ`. Del file originale non contiene nulla.

2. Esecuzione all'indietro. Il programma esegue `uncall comprimi(...)` su ogni blocco, di nuovo in un `par`. L'inverso di una sequenza è la sequenza degli inversi in ordine opposto, quindi gli stadi si disfano dall'ultimo al primo:

- **gamma**: `call corsa` conta gli zeri in cima fino all'1, poi il raddoppio (inverso del dimezzamento) ricompone x dal bit più significativo;
- **RLE degli zeri**: se in cima c'è uno 0, sotto c'è una lunghezza e si rispingono gli zeri; il testimone si ricalcola guardando la cima, e l'asserzione d'uscita dell'`if`, diventata guardia d'ingresso, sceglie il ramo;
- **MTF**: `call rigioca` ricostruisce la tabella finale da `m`; poi, simbolo per simbolo dall'ultimo, `s[i] += tab[0]`, lo spostamento disfatto e la posizione cancellata;

- **BWT**: `call bwt_inversa` ricostruisce il blocco da `out` e `idx`; poi l'ordinamento rifatto e `uncall estrai`, che cancella `out` e `idx` verificando che fossero davvero la trasformata di quel blocco; infine l'ordinamento disfatto;
- **RLE iniziale**: quattro valori uguali in cima identificano una corsa, che si riespande.

Nessuna di queste procedure è scritta: sono gli inversi, calcolati dalla VM istruzione per istruzione.

3. Scrittura. Dopo l'`uncall` l'array `blocco` contiene i byte originali. Il programma li sposta su uno stack d'uscita (`call scarica`) perché gli array, alla fine del `main`, devono essere a zero; l'involucro li scrive subito in `file.txt` e passa al gruppo successivo; dopo l'ultimo frame controlla il CRC dei dati originali. Più file `.kgz` concatenati si decomprimono nella concatenazione dei dati, come in gzip. In compressione, se il file d'ingresso va rimosso (senza `-k`), il comando decomprime prima il risultato, sempre in streaming, e lo confronta pezzo per pezzo con l'originale; solo allora rimuove l'ingresso.

4. Verifica: `kgzip2 -t`. `-t` decomprime senza scrivere nulla, controllando il CRC di ogni frame e alla fine quello dei dati. Il CRC è necessario: le asserzioni d'uscita, all'indietro, diventano controlli sullo stato, ma un file alterato non porta per forza a un'asserzione violata. In una prova con un bit invertito, prima di introdurre il CRC, la decompressione non è terminata. Con il CRC, 40 file con un bit invertito a caso (20 per algoritmo) sono stati rifiutati tutti in meno di un decimo di secondo ciascuno; la suite di test, su un file a più frame, prova anche ogni troncamento e un byte alterato in ogni posizione, e li rifiuta tutti. Che cosa fa invece l'`uncall` quando il CRC non lo ferma è misurato in §5.6: quasi sempre si ferma da solo.

4 Ordinare le rotazioni

È la parte costosa della BWT, e quella su cui [1] chiede miglioramenti. Tutti gli ordinamenti di questa sezione risolvono le parità con l'offset minore: producono lo stesso `offs`, la stessa colonna e lo stesso `idx`, e sono intercambiabili senza toccare il formato.

Il punto di partenza: bubble sort, $O(n^3)$. Il capitolo 4 compie $(n-1)^2$ confronti, ciascuno su tutti gli n caratteri, perché la guardia di un `if` non distinguerebbe "entrato senza trovare" da "non entrato".

Uscita anticipata nel confronto. Con `||` nelle guardie l'uscita anticipata si fa sul ciclo invece che sull'`if`. Il testimone `pref` (lunghezza del prefisso comune) vale esattamente k finché le rotazioni coincidono:

```
from k == 0 do
    ... c1, c2 = k-esimi caratteri ...
    if c1 > c2 then gt += 1 fi gt == 1
    if c1 == c2 then pref += 1 fi pref > k
    k += 1
loop until (k > pref) || (k == n)
if pref == n then k -= n else k -= pref  k -= 1 fi pref == n
```

Lo stato finale dice da solo quante volte si è girato ($k = \texttt{pref}+1$ oppure n), ed è ciò che serve per riportare k a zero; all'indietro il ciclo riparte da lì e rifà lo stesso numero di giri. Un confronto costa il prefisso comune più uno. Insieme al passo interno accorciato (le ultime i posizioni sono già a posto) dà la variante *bolle con uscita anticipata*.

Merge sort reversibile. `fondi` fonde due metà ordinate in un array d'appoggio e le riporta indietro con scambi. Ogni scelta d (1 se si prende da destra) va su `scarto`; a parità si prende da sinistra, quindi è stabile. I confronti sono $O(n \log n)$, ma ciascuno costa il prefisso comune: sui dati ripetitivi si torna a $O(n^2 \log n)$.

Raddoppio dei prefissi, $O(n \log n)$. L'idea di Manber e Myers [3]: se $r_h[i]$ è il rango della rotazione i sui primi h caratteri, ordinare per la coppia $(r_h[i], r_h[i+h \bmod n])$ dà l'ordine sui primi $2h$. All'inizio $h = 1$ e il rango è il byte. Ogni giro:

1. ordina le coppie con due **counting sort stabili**, prima sulla seconda chiave e poi sulla prima, partendo dall'ordine $0, \ldots, n-1$: a parità vince l'offset minore. Ogni passata è pulita: istogramma e cumulata si disfano, e le posizioni libere `pos[c]`, che alla fine valgono cumulata più occorrenze, tornano alla cumulata togliendo l'istogramma;
2. assegna i ranghi nuovi scorrendo l'ordine: il contatore cresce quando la coppia cambia, e la guardia d'uscita lo confronta con il rango appena dato alla rotazione precedente (`fi q > rn[b]`);
3. se $2h \geq n$, o se i ranghi sono già tutti distinti, l'ordine è definitivo; altrimenti si ricorre con $2h$ scambiando i ruoli dei due array di ranghi.

Due dettagli rendono il ciclo reversibile ed efficiente. L'*indicatore di arresto* va su `scarto`: dopo la ricorsione i ranghi con cui era stato calcolato sono stati consumati, e all'indietro non lo si potrebbe ricalcolare. E i dati di un giro concluso (ranghi vecchi, ordine intermedio, ordine del giro) vanno su `scarto` invece di essere ricalcolati all'indietro: una passata $O(n)$ invece di tre counting sort, al prezzo di al più $3n$ interi per giro. È il compromesso fra spazio e tempo di Bennett; sul testo dimezza il costo dell'ordinamento (§5). Ogni giro costa $O(n + n_r)$, con $n_r = \max(n_1, 256)$ chiavi; i giri sono al più $\lceil \log_2 n \rceil$, e sui dati poco ripetitivi pochi.

5 Valutazione

Ambiente. Kairos al commit `1aa59fb` (15 ottobre 2026), VM compilata con `gcc 13.3 -O3 -march=native -flto`, Python 3.13; macchina virtuale con 2 vCPU Intel Xeon e 8 GB. Tempi in secondi; per gli ordinamenti è il minimo di tre esecuzioni della sola VM (escluso il frontend). Il riferimento è bzip2 1.0.8 con `-9`. Le macchine a 2 core limitano le misure di concorrenza; il capitolo 4 usava 6 core.

Correttezza. Ogni uscita è controllata contro un riferimento Python indipendente: per la BWT, ordinamento stabile delle rotazioni; per kgzip2, un modello della pipeline bit per bit (`modello.py`). La BWT con le quattro varianti dell'ordinamento è stata confrontata con il riferimento su 300 casi (byte casuali con alfabeto fino a 256 simboli, testo, blocchi periodici, costanti, alfabeto di tre simboli; n fino a 400): nessuna discordanza, e le quattro varianti danno lo stesso `offs`, la stessa colonna e lo stesso `idx` anche sui blocchi periodici, dove molte rotazioni sono identiche. La suite di test della repository (`test/test_completo.py`, 367 controlli, nessuno fallito) comprime e decomprime con entrambi gli algoritmi i 9 file della Tabella 4 e 13 casi limite (file vuoto, di 1 e 2 byte, corse di esattamente 4, corse di 300 byte, quasi-corse, tutti i 256 byte, due blocchi in `par`): l'uscita compressa coincide byte per byte con il modello, il ritorno con l'originale, e ogni file viene poi alterato e troncato per controllare che sia rifiutato. Prova anche lo streaming: un file a più frame compresso con gruppi diversi dà gli stessi byte, si decomprime con gruppi diversi e concatenato ad altri flussi, rifiuta i dati estranei in coda, e su un file a più frame ogni troncamento e un byte alterato in ogni posizione vengono rifiutati. Infine i comandi veri (rimozione dell'ingresso dopo la verifica, `-k`, `-c`, `-T`, filtro da stdin a più blocchi, flussi concatenati da una pipe, `-t` su file alterati e troncati).

5.1 Ordinamento delle rotazioni

Il programma di misura carica un blocco, esegue `ordina` ed `estrai`, stampa e poi disfa tutto con `uncall`: il tempo comprende quindi un ordinamento in avanti e uno all'indietro. Quattro tipi di dati: *casuale* (byte uniformi), *testo* (finestre del README di Kairos), *periodico* (periodo 6, i blocchi di `genera_bwt.py` del capitolo 4) e *costante*. Gli ultimi due sono il caso peggiore per chi confronta caratteri: tutte le rotazioni condividono prefissi lunghi.

	periodico				costante			
n	bolle	bolle+u.a.	fusione	raddoppio	bolle	bolle+u.a.	fusione	raddoppio
16	0,37	0,04	0,02	0,03	0,50	0,18	0,05	0,03
32	3,20	0,32	0,07	0,04	4,26	1,42	0,24	0,04
64	27,43	3,00	0,37	0,07	34,86	11,56	1,18	0,07
128	–	–	1,46	0,11	–	–	5,27	0,11
256	–	–	7,82	0,20	–	–	–	0,20
512	–	–	–	0,45	–	–	–	0,45
1024	–	–	–	1,01	–	–	–	0,99
2048	–	–	–	2,20	–	–	–	2,23
4096	–	–	–	4,88	–	–	–	4,81
8192	–	–	–	10,47	–	–	–	10,61

Tabella 2: Tempo (s) di ordinamento in avanti e all'indietro, dati ripetitivi. "–" = non misurato (oltre i 30 s per esecuzione, o oltre le dimensioni utili).

	casuale				testo			
n	bolle	bolle+u.a.	fusione	raddoppio	bolle	bolle+u.a.	fusione	raddoppio
16	0,33	0,02	–	–	–	–	–	–
32	2,84	0,06	0,02	0,01	–	0,07	–	–
64	22,65	0,25	0,06	0,01	–	0,29	0,06	0,02
128	–	1,01	0,13	0,02	–	1,44	0,15	0,07
256	–	4,30	0,30	0,03	–	6,03	0,37	0,09
512	–	–	0,70	0,12	–	–	0,88	0,22
1024	–	–	1,59	0,24	–	–	2,10	0,45
2048	–	–	3,73	0,47	–	–	5,17	1,10
4096	–	–	8,21	0,96	–	–	11,78	2,65
8192	–	–	–	1,89	–	–	–	5,22

Tabella 3: Tempo (s) di ordinamento in avanti e all'indietro, dati casuali e testo.

Sui dati periodici le pendenze fra gli ultimi punti misurati sono 3,15 per il bubble sort (il cubo atteso, come nel capitolo 4), 3,23 con l'uscita anticipata (che non aiuta: i prefissi comuni sono lunghi quanto il blocco), 2,24 per il merge sort ($n^2 \log n$) e 1,13 per il raddoppio ($n \log n$). A $n = 64$ il bubble sort impiega 27,4 s e il raddoppio 0,07 s; il merge sort a $n = 256$ ne impiega 7,82, il raddoppio 0,20. Il raddoppio ordina un blocco periodico di 8192 byte in 10,5 s, ed è l'unico che ci arriva.

Sui dati poco ripetitivi l'uscita anticipata basta già a rendere il confronto quasi costante, e il merge sort cresce con pendenza 1,19 sui casuali e 1,25 sul testo. Il raddoppio resta comunque il più veloce (pendenza 1,00 sui casuali, 1,19 sul testo), perché si ferma appena i ranghi sono distinti: a $n = 4096$ casuale 0,96 s contro 8,21, sul testo 2,65 s contro 11,8. Ha un costo fisso per giro legato alle $n_r \geq 256$ chiavi del counting sort, che pesa sui blocchi piccoli.

5.2 Compressione

Sul testo kgzip2 riduce i file a 44,0%–47,9%, contro 39,2%–42,4% di bzip2. Sui dati molto regolari fa meglio di bzip2: l'immagine 64×64 scende a 2,8% contro 3,0%, il file periodico a 0,7% contro 0,7%, gli zeri a 0,6% contro 1,0%; qui conta anche l'intestazione più corta. Sul testo ripetuto due volte bzip2 sfrutta meglio la ripetizione (26,6% contro 35,8%); sui byte casuali il blocco memorizzato tiene il file a 100,4%, contro 111,8% di bzip2. La colonna "ordine 0" mostra quanto si guadagnerebbe sostituendo il codice gamma con un codice adattivo: sul testo quattro o cinque punti, quasi tutta la distanza da bzip2.

5.3 Dove va il tempo

La BWT, ordinamento compreso, è il 67% del tempo. Dentro `bwt` l'ordinamento si esegue due volte (`call` e `uncall`), e altrettante nel verso opposto: è il prezzo della pulizia. Nella prima versione del

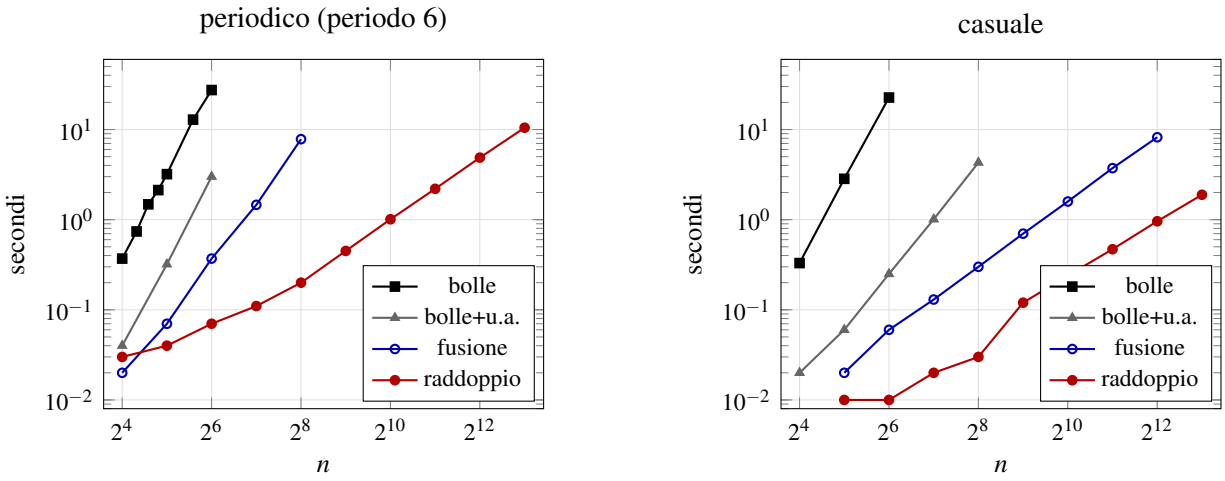


Figura 2: Tempo di ordinamento (andata e ritorno) in scala log-log. Una legge di potenza appare come una retta e la pendenza ne è l'esponente.

file	byte	bzip2 -9	kgzip2	%	bit/byte	ordine 0	comprimi	`uncall`
tesi (LaTeX)	8192	3343	3816	46,6	3,73	41,7	12,5	11,4
Janus.c	8192	3214	3601	44,0	3,52	39,6	12,3	11,1
README	8192	3473	3928	47,9	3,84	43,0	13,0	11,6
testo ripetuto 2 volte	8192	2182	2934	35,8	2,87	31,9	21,8	21,3
periodico	8192	61	58	0,7	0,06	0,1	18,4	18,5
immagine 64×64	4109	124	116	2,8	0,23	1,7	3,0	3,1
immagine 16×16	269	78	63	23,4	1,87	11,3	0,3	0,3
zeri	4096	43	25	0,6	0,05	0,0	0,8	0,8
casuale	4096	4580	4111	100,4	8,03	99,8	23,1	0,0

Tabella 4: Blocchi da 8192 byte. "%" è la dimensione compressa rispetto all'originale; "ordine 0" è la dimensione che avrebbero gli stessi gettoni con un codice entropico ideale d'ordine 0 (senza intestazioni). Tempi della VM in secondi (il frontend, che ora traduce solo la parte fissa, resta sotto il decimo di secondo). In tutte le righe il ritorno è identico byte per byte e l'uscita compressa coincide con il modello.

raddoppio, che ripuliva ogni giro ricalcolandolo, le due righe della BWT costavano 33,2 s; mettendo su `scarto` i dati dei giri conclusi scendono a 16,1 s.

5.4 Concorrenza e costo dell'inverso

I blocchi sono indipendenti e il `par` li esegue su thread diversi in entrambi i versi: anche la decompressione è parallela senza che nessuno l'abbia scritta, perché l'inverso di un `par` è il `par` degli inversi. Con 2 core il guadagno non può superare 2×; il capitolo 4 ha già mostrato, su 6 core, che il `par` raggiunge il 91% del throughput di processi indipendenti.

Quanto al costo dell'inverso: sui file della Tabella 4 con almeno un secondo di compressione, l'`uncall` costa in media 0,81 volte la compressione nella VM. Non è un decodificatore più lento: ripete le stesse operazioni in ordine opposto. Nella prima versione i bit passavano dal parser, una riga ciascuno, e la decompressione degli 8 KiB della tesi pagava 5 s di frontend in più; con i dati scritti direttamente nel bytecode quel costo è sparito.

5.5 Streaming e memoria

Prima dello streaming l'involucro leggeva tutto il file e metteva tutti i blocchi in un solo programma: la memoria cresceva con il file, di circa 1020 byte di RAM per byte d'ingresso (Python e VM insieme, di cui 683 nella VM), da 78 MiB per 16 KiB a 126 MiB per 64 KiB, e a 256 KiB non riusciva affatto: con più di

stadio	secondi	quota
RLE iniziale	2,38	10%
BWT: ordinamento	7,39	31%
BWT: estrazione, inversa, pulizia	8,71	36%
move-to-front	3,00	13%
RLE degli zeri	0,97	4%
Elias gamma	1,49	6%
totale	23,94	100%

Tabella 5: Costo per stadio su un blocco di 8192 byte della tesi, andata e ritorno (`call` e `uncall` dello stadio). La seconda riga della BWT comprende l'ordinamento disfatto e rifatto dentro `bwt`.

	comprimi (s)			`uncall` (s)		
blocchi	sequenziale	`par`	guadagno	sequenziale	`par`	guadagno
2 da 4096	11,8	6,2	1,89×	10,6	5,5	1,93×
4 da 2048	12,4	6,7	1,86×	10,7	5,8	1,84×

Tabella 6: Gli 8 KiB della tesi divisi in blocchi, come rami di un `par` o uno dopo l'altro. Stesso programma, stessi byte prodotti; tempi della VM, macchina a 2 core.

10 blocchi in un programma i nomi per blocco si sovrapponevano (§3). Con lo streaming, da 16 a 256 KiB il picco resta fra 52 e 56 MiB per Python e fra 42 e 44 MiB per la VM (Tabella 7): dipende dal gruppo, non dal file. Il tempo invece resta lineare: 1,1 KiB al secondo in compressione e 1,1 in decompressione su 2 core, verifica esclusa. Un file da 100 GB ora si comprime con poche decine di MiB, ma richiederebbe circa 1053 giorni: la memoria non è più il limite, il tempo sì.

5.6 Un file alterato e l'`uncall`

Il CRC di ogni frame ferma un file alterato prima della VM. Che cosa succederebbe senza? L'esperimento (`bench/manomissione.py`) comprime 1 KiB della tesi, lo altera e lancia direttamente `uncall comprimi`, saltando il CRC (lo stesso fa il comando con `--salta-crc`): 200 bit invertiti a caso, 50 coppie di bit adiacenti alterate, ogni contatore del frame cambiato di ± 1, un bit in più o in meno in fondo. Una decompressione normale dura 1,3 s; oltre 13 s conta come non terminata (Tabella 8).

Ne esce una proprietà che un decompressore ordinario non ha: quasi sempre la VM si ferma da sola. L'inverso deve disfare esattamente il calcolo del compressore, e un file che il compressore non avrebbe mai prodotto lascia una traccia: un ausiliario che non torna a zero (`delocal`), un'asserzione d'uscita che contraddice il ramo, una `pop` su uno stack vuoto. Nessuno di questi controlli è stato scritto per validare l'ingresso: sono le condizioni che rendono il programma invertibile, lette all'indietro. Ogni invariante del formato (corse massimali dell'RLE iniziale, una corsa di zeri sempre annunciata dal suo 0, il cammino della BWT che deve tornare su `idx`, il numero esatto di bit) diventa un controllo gratuito: in kgzip2 sono questi a fermare il 83,1% delle alterazioni.

La prima misura ha trovato però un difetto: il 25,4% delle alterazioni faceva girare kgzip2 all'infinito. La causa era `corsa`, che estende una corsa finché la cima dello stack è uguale al simbolo, con guardia d'ingresso `run == 1`. All'indietro la guardia d'ingresso diventa la condizione d'uscita, e nella RLE degli zeri l'inverso parte dalla lunghezza letta dal file e spinge zeri finché `run` torna a 1. Se l'alterazione fa leggere una corsa lunga 0, `run` scende a $-1, -2, \ldots$ senza mai valere 1, e la VM spinge zeri per sempre. La correzione è una riga: `from run <= 1`. Sui dati validi `run` parte da 1 e cresce, le due guardie coincidono nei due versi e il file compresso non cambia; su quelli alterati l'inverso esce comunque, e il `delocal` di `run` in `rle0` ferma la VM. Sulle stesse prove le non terminazioni passano da 66 a 0, tutte diventate asserzioni, e la suite di test controlla che nessun bit invertito faccia girare l'`uncall` all'infinito. La regola vale per ogni ciclo reversibile: una guardia d'uguaglianza su un contatore che arriva dai dati va

		prima (MiB)		streaming (MiB)		secondi	
KiB	byte compressi	Python	VM	Python	VM	comprimi	`uncall`
16	7826	52	26	52	42	14	12
64	31232	68	58	56	44	55	51
256	101202	non riesce		56	44	238	223

Tabella 7: Picco di memoria residente al crescere del file (sorgenti di Kairos concatenati), per il processo Python e per il processo della VM; "prima": la versione che leggeva tutto il file e metteva tutti i blocchi in un solo programma (solo compressione). Il processo della VM nasce da un `fork` dell'involucro, e il suo picco comprende le pagine condivise con Python. Blocchi da 8 KiB, gruppi di 2, 2 core.

	prove	VM fermata da sola	da controlli di reversibilità	non termina	uscita sbagliata
kgzip2, prima della correzione	260	71,2%	57,7%	25,4%	3,5%
kgzip2	260	96,5%	83,1%	0,0%	3,5%
klzma	254	98,4%	3,1%	0,0%	1,6%

Tabella 8: `uncall comprimi` su un blocco di 1 KiB della tesi alterato, senza il controllo del CRC. Le prove sono le stesse nelle due righe di kgzip2. "Da controlli di reversibilità: `delocal` e asserzioni d'uscita, che esistono solo perché il programma deve potersi invertire; il resto della colonna precedente sono `pop` da stack vuoti e indici fuori dai limiti. "Non termina": oltre dieci volte il tempo normale.

scritta come disuguaglianza monotona, perché all'indietro è lei a dover fermare il ciclo.

Restano le uscite sbagliate senza errori, il 3,5% in kgzip2 e il 1,6% in klzma: alterazioni che portano a un altro file che il compressore avrebbe potuto produrre davvero. Il caso più netto è `idx` cambiato di uno, che passa sempre: la BWT è una biiezione fra $(\texttt{out}, \texttt{idx})$ e i blocchi, quindi un altro indice decodifica un altro blocco perfettamente coerente. È un limite di principio: un compressore toglie ridondanza, e dove non ce n'è non c'è nulla da controllare. Per questo i CRC restano: quello dei frame ferma tutto prima della VM, quello dei dati copre i casi che la reversibilità non può vedere. In klzma, infine, la VM si ferma quasi sempre (231 casi su 254) per un indice fuori dai limiti, cioè per lo stesso controllo che farebbe un decodificatore LZ scritto con cura: i controlli propri della reversibilità lì contano poco.

6 Limiti

- **Codice entropico.** Il codice gamma è fisso: non si adatta alle frequenze del blocco. È la ragione principale della distanza da bzip2, che usa Huffman con tabelle per gruppi di simboli. La colonna "ordine 0" della Tabella 4 dà il limite che un codice adattivo d'ordine 0 raggiungerebbe sugli stessi gettoni.
- **Dati incomprimibili.** Sui byte casuali MTF produce valori uniformi e il codice gamma spenderebbe circa 13 bit per byte; il blocco memorizzato limita il danno all'intestazione e ai CRC.
- **Velocità assoluta.** Anche con l'ordinamento $O(n \log n)$ kgzip2 elabora qualche centinaio di byte al secondo per core: è un interprete di un linguaggio reversibile, e ogni passo in avanti ne implica uno all'indietro per pulire. Con lo streaming la memoria non limita più la dimensione dei file; il tempo sì (§5.5).
- **Spazio.** Lo stack `scarto` tiene fino a circa $3n\lceil \log_2 n \rceil$ interi durante l'ordinamento; la pulizia avviene prima della fine di `bwt`.
- **Blocchi piccoli.** Blocchi da 8 KiB contro i 900 KiB di bzip2: la BWT rende di più su blocchi grandi.

7 Lavori futuri

- **Huffman canonico reversibile**, con la tabella delle lunghezze trasmessa nel blocco e ricalcolata per la pulizia con una coppia di procedure, o una codifica aritmetica a interi.
- **SA-IS** [8], lineare, al posto del raddoppio: più difficile da rendere pulito, ma toglierebbe il fattore $\log n$.
- **Blocchi più grandi**: con i dati caricati come bytecode il limite non è più il frontend ma il tempo della VM, lineare a meno del $\log n$ dell'ordinamento.
- **Pipeline concorrente fra stadi**: oggi il parallelismo è fra blocchi; con i canali di Kairos gli stadi potrebbero lavorare come processi in catena, uno per stadio.
- **Uso nella tesi**: la tabella dei costi del capitolo 4 può riportare i tre ordinamenti, e la BWT pulita in senso stretto chiude uno dei tre limiti dichiarati.

8 Conclusioni

Un compressore reversibile completo si scrive una volta sola: `call comprimi` comprime, `uncall comprimi` decomprime, e i `par` fra blocchi valgono nei due versi. La condizione è la pulizia di ogni stadio, e il lavoro mostra che si ottiene con poche tecniche ricorrenti: la coppia di procedure per la BWT, i testimoni azzerati guardando lo stato, la spazzatura su uno stack ripulita disfacendo chi l'ha prodotta, e formati pensati per essere letti dalla cima, cioè nell'ordine in cui li consuma l'inverso. Il punto che Lyngby et al. indicavano come aperto, l'ordinamento delle rotazioni, si chiude con il raddoppio dei prefissi: $O(n \log n)$ qualunque siano i dati, contro l'$O(n^3)$ del capitolo 4. La distanza da bzip2 che resta è quasi tutta nel codice entropico, il prossimo stadio da rendere reversibile.

Riferimenti bibliografici

[1] M. Lyngby, J. Nylandsted, R. Glück, T. Yokoyama. *Towards Clean Reversible Lossless Compression: A Reversible Programming Experiment with Zip*. Reversible Computation (RC 2024), LNCS 14680, pp. 94–102.

[2] M. Burrows, D. J. Wheeler. *A Block-sorting Lossless Data Compression Algorithm*. SRC Research Report 124, Digital Equipment Corporation, 1994.

[3] U. Manber, G. Myers. *Suffix Arrays: A New Method for On-Line String Searches*. SIAM Journal on Computing 22(5), 1993.

[4] C. H. Bennett. *Logical Reversibility of Computation*. IBM Journal of Research and Development 17(6), 1973.

[5] T. Yokoyama, R. Glück. *A Reversible Programming Language and its Invertible Self-Interpreter*. PEPM 2007.

[6] P. Elias. *Universal Codeword Sets and Representations of the Integers*. IEEE Transactions on Information Theory 21(2), 1975.

[7] J. Seward. *bzip2 and libbzip2*, versione 1.0.8, 2019.

[8] G. Nong, S. Zhang, W. H. Chan. *Linear Suffix Array Construction by Almost Pure Induced-Sorting*. Data Compression Conference (DCC), 2009.

A File e comandi

Tutto è nella repository `kairos-compressori`, con Kairos come sottomodulo in `kairos/`.

percorso	contenuto
`kgzip2/kgzip2.kairos`	la libreria: tutti gli stadi e `comprimi`
`kgzip2/driver.py`	il programma di un gruppo di blocchi, dati come bytecode, contatori dei frame
`kgzip2/modello.py`	la stessa pipeline in Python, bit per bit
`kgzip2/fusione.kairos`	confronto a uscita anticipata e merge sort (passo intermedio)
`kgzip2/ordinamenti.py`	le quattro varianti dell'ordinamento: verifica e tempi
`kgzip2/profilo.py`	costo per stadio, con programmi troncati
`comune/kvm.py`	esecuzione dei programmi Kairos, cache del frontend
`comune/contenitore.py`	il formato a frame e i CRC, comune ai due algoritmi
`comune/flusso.py`	lo streaming a gruppi di blocchi
`comune/cli.py`	la riga di comando
`test/test_completo.py`	la suite di test (367 controlli)
`bench/misura.py`	il confronto con bzip2, xz e LZMA
`bench/memoria.py`	memoria e tempo al crescere del file
`dist/costruisci.sh`	gli eseguibili autonomi `dist/bin/kgzip2` e `dist/bin/klzma`

```
make eseguibili                       # dist/bin/kgzip2, dist/bin/klzma
kgzip2 -v file.txt                    # file.txt.kgz, poi rimuove file.txt
kgzip2 -d file.txt.kgz                # uncall dello stesso programma
kgzip2 -t file.txt.kgz                # controlla CRC e decompressione, non scrive
tar c cartella | kgzip2 > cartella.tar.kgz   # streaming, memoria costante
kgzip2 -T 1 -v grande.bin             # un blocco alla volta, con avanzamento
make test                             # 367 controlli
python3 kgzip2/ordinamenti.py tempi raddoppio periodico 256 1024 4096
```